

Project Proposal: Rubik's Cube PLL Solver

Joseph Fedoronko
Ann Arbor, United States
<https://github.com/fedoronj>
fedjz88@gmail.com

Abstract—Artificial intelligence and computer vision have already begun to expand to areas such as automation, prediction, and decision making; an area that could have the greatest potential is in enhancing human learning. This project proposal explores how this idea can be applied to learning new skills or perfecting existing ones. One such task, the solving of a Rubik's cube as fast as possible, requires the use of memorization of many different algorithms for multiple steps. In this project, we will focus on the last step of an advanced method: PLL (permutation of the last layer). This method employs 21 different algorithms for each of the 21 different patterns that can be seen along the 12 squares that make up the top layer of the cube. In order to identify the given case in as little time as necessary, speed cubers must identify the case whilst only looking at 2 sides of the cube, or 6 total squares. The Rubik's Cube PLL trainer aims to work alongside the solver, analyzing the cube's current state and generating the correct algorithm needed for the given case. The end result allows the user to guess what case they are seeing, then consult with the trainer, which will have an algorithm and name displayed based on the cube's current orientation. Due to the nature of speed cubing, the user will only look at 2 sides of the cube when identifying the given case. Therefore, the trainer must similarly use the remaining 2 sides of the cube to determine what case the user is looking at. It follows that two sets of instructions can be generated that cover the 21 different algorithms. This project offers a classical application of both computer vision and deep learning theory. In order to begin logic to determine which case is present, the model must first accurately extract a 45 element array consisting of the colors of each square on the cube, without the bottom 9. From there multiple different methods can be used to convert this matrix into an algorithm. Firstly, traditional logic can be used with hard coded values representing each individual pattern that can occur. Another method involves training a simple neural network to recognize groupings of colors using cost functions with neighboring stickers. This model would be trained on all possible color orientations for each PLL case that could occur. From there, the model would be able to comprehend which case is represented by the matrix and the generate the corresponding algorithm and name to display to the user.

I. PROPOSED METHODOLOGY

In order to obtain real time processing for our model (under 2 seconds), there was a need for a lightweight and efficient model. In order to achieve the goals set out for this project, it was proposed that a computer vision system is used to handle object detection tasks: sticker and color recognition. Further, a neural network was proposed to handle the classification of color patterns based on trained data. The computer vision system would employ OpenCV functions such as canny edge detection and convex hull algorithms to properly separate a colored sticker from the surrounding cube border. With these

pixels properly categorized, the end result from the computer vision system would be three 3x3 matrices, representing the top, left, and right hand sides of the cube, according to the camera's field of view. Using these matrices, the first check would be to see what stage of solving the cube the user is currently at. For example, using the bottom 2 rows from each matrix, we can compare the color values to one another, with the same color indicating that the user has solved the bottom 2 layers of the cube. Similarly the top of the cube can be checked to further determine the cube's stage. A full matrix of yellow values would indicate that the user has completed the stage of orienting the last layer (OLL). Combining this information with the results of the bottom 2 layers of the cube, we are able to determine whether the system is ready to start processing the remaining patterns of 6 colors to determine the PLL case. This would result in a state change for the model and would then trigger to run the 6 colors through the neural network, matching the color pattern with the ones that it was trained on. Due to the limitation that CNNs can struggle in tasks involving robust object detection and fine-grained color classification, a hybrid approach was adopted to leverage the complementary strengths of multiple machine learning methods and achieve improved overall performance. Furthermore, deep learning networks excel at classifying patterns based on trained data, which is easily generated with different combinations of cube orientations, allowing for quick response times. This method also reduces the need to create in depth logic to classify a specific case based by iterating over each individual square.

II. IMPLEMENTATION

Once the methodology was laid out, the first task was the implementation of object detection via OpenCV. This process is similar to most computer vision projects, which involve the input of an image and applying functions to handle filters, masks, and math operations. An initial hue, saturation, and value (HSV) range was determined for each color with the goal of removing edge cases and mislabeling. Next, a mask was created that shows the detected colors for each pixel in HSV format. Then, filtering of the mask (removing light and noise), followed by detecting the contours which represent the actual stickers with black borders. From there, centroids were computed that represent the contours by sorting, organized into 3x3 grids, listing the color of each sticker. The average pixel value for a designated sticker area was assigned to a singular color. This final value after averaging and filtering is

fed through logic that classified the sticker color based on the pre-determined HSV value.

III. CHALLENGES

Along the way, there were many shortcomings in this project that ultimately contributed to this project being incomplete. Originally, it was believed that this project could be completely entirely using neural networks. The thought process was to use a pre-existing model to classify the color of each sticker on the cube, then feed that data through another neural network to handle pattern recognition. This plan suffered because this type of color detection had not previously been done in a way that was accessible and easy to implement. To address this challenge, a small dataset with manually created labels was developed with the goal of training a model in Roboflow capable of extracting color information from cube faces. The issue with this approach was that there were very limited results due to using a model trained on less than 30 images with corresponding bounding boxes; see appendix A. Future research led to better understanding of the limitations of CNN based system and color detection-leading in turn to computer vision applications [1]. Using these systems, I was able to achieve better results than with either of the first 2 approaches. However, the issue that persisted was that colors were commonly misidentified, such as whites and greens labeled as yellows and blues respectively. One of the most challenging aspects of the computer vision approach was the fine tuning that was required for the nominal HSV color values. These values were selected based on previous research and were initially expected to perform adequately without further adjustments. As a result of my cube's specific color shades, as well as lighting, these values shifted vastly with multiple outliers per side, which did not leave room for fine tuning nominal values.

Without the use of premium features such as AI-assisted labeling via Roboflow, NVIDIA powered GPUs, this project hit many roadblocks that might have been avoidable with better hardware. Going forward, I would suggest that computer vision be maintained as the go to method for sticker detection and color classification. With a more robust approach to filtering and masking, I believe that the data can be accurately extracted with solid repeatability. From that point on, a simple neural network is proposed to handle the classification of the PLL case as previously proposed. This network would eliminate the need to create complicated logic to determine the specific case presented. Inspiration for this network could take the form of

IV. FUTURE WORK

1) *Object Detection*: Future work on this project would start with developing a robust object detection strategy to identify objects. This process will specifically focus on achieving 95% accuracy with respect to both sticker colors and boundaries. Only once this metric is reached will we be able to fully test PLL specific case logic. With the matrix output

of the object detection task set, the next step is to valid case by case logic for each of the 21 distinct PLL cases.

2) *Logic Validation*: Although the process of creating custom case logic for each of the 21 cases may be a tedious task, it provides a way to debug the existing logic early on, reducing the time and complexity involved in troubleshooting the deep learning pattern logic later. This step also serves as a proof of concept for the entire project. Once implemented, the product will be functioning as intended and any future modifications to the logic and framework would only serve to increase the performance of the system.

3) *Deep Learning Pattern Detection*: Employing deep learning for the last step of this project offers a more streamlined and efficient approach compared to complicated logic statements. For the sake of readability and legacy code, this step would serve to allow any user to understand what is happening behind the scenes with greater knowledge compared to the traditional logic approach. Additionally, future modifications would mainly involve re-training the model to recognize different patterns, potentially allowing for adaptation of the product to the entire Rubik's cube solving process rather than just the last step.

V. CONCLUSION

This project provided interesting insight into the different approaches to object detection problems. In addition, this project explores the unique union of computer vision and deep learning, something that is mainstream in the machine learning industry today. Models such as YOLO [2] demonstrate how deep learning can learn complex visual features directly from image data, while traditional computer vision techniques remain valuable for image preprocessing, data augmentation, and result refinement. Through the development of both computer vision and deep learning based solutions, this project highlighted the tradeoffs between handcrafted feature extraction and data-driven learning approaches, as well as the importance of dataset quality, model selection, and environmental factors such as lighting and image consistency.

VI. APPENDIX A

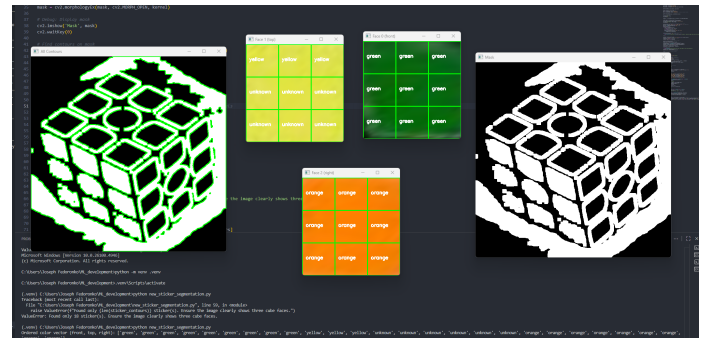


Fig. 1. Sample computer vision output

REFERENCES

- [1] Akito Morita, Io Yamada, and Hirotugu Okuno. Effects of color constancy algorithms on robustness of cnns to changes in illumination conditions. In *2024 SICE Festival with Annual Conference (SICE FES)*, pages 1090–1096, 2024.
- [2] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection, 2016.